



CSE335s: Operating Systems

Project Report

(CPU Scheduler)

Team Members	ID
Arsany Hany Anwar	2200655
Andrew Ehab Tharwat	2200249
Alaa Abdel Hakem Mahmoud	2200498
Rola Ahmed Kassem	2200658
Radwa Yasser Ahmed	2200239
Yomna Ahmed Farag	2200224
Ziad Ahmed Orabi	2200399

Contents

Table of Contribution	5
Chapter 1: Introduction	6
Project Idea	6
Project Description	6
Design	6
Implemented Key Features.....	6
System Specifications	6
Chapter 2: System Diagram	7
Chapter 3 : Algorithms	8
First Come First Served (FCFS).....	8
Abstract	8
Algorithm	8
Code	8
Priority Non-Preemptive Scheduling Algorithm.....	9
Abstract	9
Algorithm	9
Code	10
Shortest-Job First (Preemptive)	11
Abstract	11
Algorithm	11
Code	11
Shortest-Job First (Non-Preemptive).....	12
Abstract	12
Algorithm	12
Code	12
Priority Preemptive Scheduling Algorithm.....	13
Abstract	13
Algorithm	13
Code	14
Priority Non-Preemptive Scheduling Algorithm.....	15
Abstract	15
Algorithm	15
Code	16

Round Robin (configurable time quantum)	17
Abstract	17
Algorithm	17
Code	18
Chapter 4 : Core	19
Overview	19
Scheduler.py	19
1.Import block	19
Class definition	19
2.Process adder & getter	20
3.Reset	21
4.Clock	21
5.Ready Queue	21
6.Average Timing Metrics	22
7.Process Lookup & Removal	23
8.Core Abstract Method	23
9.Heartbeat	24
Process.py	25
1. Introduction	25
2. Imports and Dependencies	25
3. Class Definition	25
4. Reset Function	26
5. Process State Management	26
6. Time Calculation Methods	26
7. Execution Mechanism	27
8. Cloning Functionality	28
9. Accessor Methods (Getters)	28
10. Helper Methods	29
11. String Representations	29
Execution.py	30
1.Constructor & Validation	30
2.Duration, Getters & Setters	30
3.Time Slice Query Methods	31
4.Execution Class Dunder Methods	31
Simulation.py	32
1. Class & Constructor	32

2. Process Management	32
Chapter 5 : GUI	35
main_window.py	35
run_controller.py	37
gantt_widget.py	39
Chapter 6 : Final Output.....	41
Priority Scheduling (regular run)	41
Dynamic Addition (inject while paused)	41
Idle Time Case	42

Table of Contribution

Name	Contribution
Arsany Hany Anwar	Algorithms
Andrew Ehab Tharwat	Algorithms
Alaa Abdel Hakem Mahmoud	GUI
Rola Ahmed Kassem	GUI
Radwa Yasser Ahmed	Core logic Report
Yomna Ahmed Farag	GUI
Ziad Ahmed Orabi	Core logic Report

Chapter 1: Introduction

Project Idea

Develop and deploy a fully functional CPU Scheduling Simulator as a graphical desktop application. It replicates an OS process scheduler's decision-making, allowing real-time observation of how different algorithms manage process execution. 1 real second = 1 simulation tick.

Project Description

Python-based desktop app modeling six algorithms:

- First Come First Served (FCFS)
- Non-preemptive Shortest Job First (SJF)
- Preemptive Shortest Remaining Time First (SRTF)
- Non-preemptive Priority Scheduling
- Preemptive Priority Scheduling
- Round Robin (configurable time quantum)

Input per process: Process ID, name, arrival time, burst time, optional priority.

Execution modes:

- *Live Mode* – 1s/tick, dynamic update of Gantt chart & remaining burst timetable.
- *At-Once Mode* – no delays, instant results.

Dynamic injection: New process can be added during simulation; its arrival time is the current simulation time.

Design

Three components:

1. **Algorithms** – Six classes inheriting from abstract Scheduler base; each implements its own process selection logic.
2. **Core Logic** – Manages simulation clock, ready queue, arrival checks, metric calculations, main tick-driven loop.
3. **GUI** – PyQt5 based; displays live Gantt chart, remaining burst table, input fields, and real-time updates.

Implemented Key Features

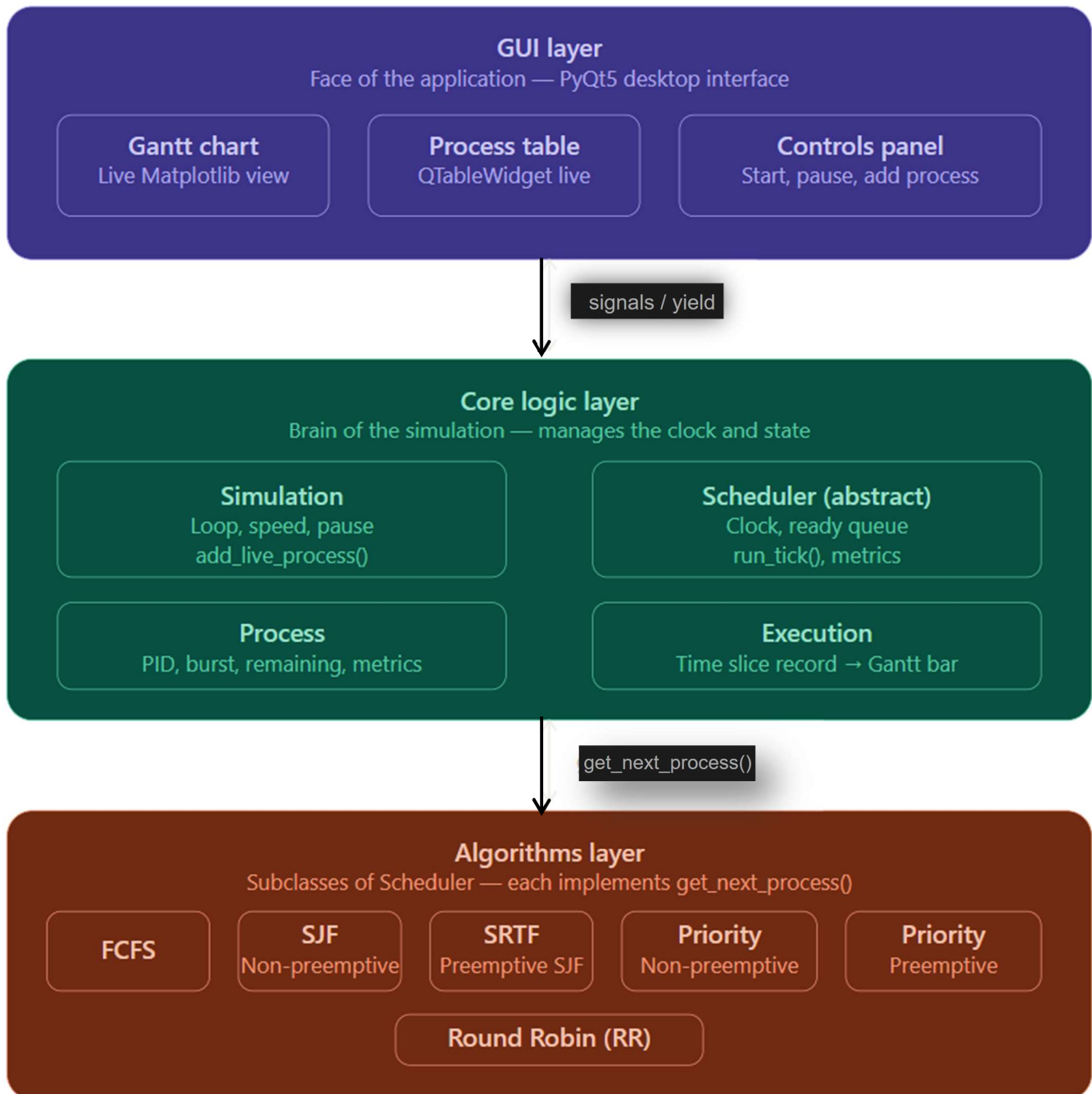
- Six scheduling algorithms (as above)
- Live Gantt chart + live remaining burst timetable (updated every second)
- Dynamic process injection while running
- Context-sensitive input fields (shown/hidden per algorithm)
- At-Once mode for instant results

System Specifications

The application is built in **Python 3.10+**, using **PyQt5** for the desktop GUI, **Matplotlib** for the embedded Gantt Chart, and **QThread** for background simulation execution. The final delivery is a standalone **Windows executable** packaged with PyInstaller, requiring no Python installation to run.

GitHub repositories: [Alaa-AHK/CPU-Scheduler](https://github.com/Alaa-AHK/CPU-Scheduler)

Chapter 2: System Diagram



Chapter 3 : Algorithms

First Come First Served (FCFS)

Abstract

FCFS (First-Come First-Served) schedules processes in the order they arrive; the first process to enter the ready queue is the first to execute. It is non-preemptive and straightforward to implement using a simple queue. Its main drawback is the convoy effect: a long process at the front of the queue blocks all shorter processes behind it, increasing average waiting time significantly.

Algorithm

1. Get all arrived, not completed processes.
2. Order them by arrival time.
 - a. If two processes have the same arrival time, order them by process id (pid).
3. Execute the first process.
4. Repeat these steps after finishing each process.

Code

```
Algorithms > FCFS.py > ...
1  from typing import List, Optional
2  from Logic.scheduler import Scheduler
3  from Logic.process import Process
4
5  class FCFSScheduler(Scheduler):
6      def __init__(self):
7          super().__init__("First-Come, First-Served (FCFS)")
8
9      def get_next_process(self, current_time) -> Optional[Process]:
10         if self.current_process and not self.current_process.is_completed():
11             return self.current_process
12
13         ready_processes = self.get_arrived_processes(current_time)
14         if not ready_processes:
15             return None
16
17         ready_processes.sort(key=lambda process: (process.get_arrival_time(), process.get_pid()))
18         return ready_processes[0]
```


Priority Non-Preemptive Scheduling Algorithm

Abstract

Priority Non-Preemptive (PNP) is a CPU scheduling algorithm that selects the process with the highest priority (lowest priority number) from all arrived processes. Once a process starts running on the CPU, it is never interrupted. It runs to completion before the next process is selected. This ensures simplicity but may cause lower-priority processes to wait longer.

Our implementation extends an abstract Scheduler class and overrides a single method `get_next_process()`, which is called every tick to decide which process runs next. It uses a helper method `_get_highest_priority()` to find the best candidate from all arrived processes and supports dynamic process addition during runtime.

Algorithm

On **every tick**, the algorithm follows these steps:

1. **Check current process:** If there is a process currently running on the CPU, check two cases:
 - **Case A — Completed:** If the process has finished (remaining time = 0) clear it from the CPU.
 - **Case B — Still Running:** If the process has not finished, keep running it. **Do not interrupt it**, even if a higher priority process has arrived.
2. **Pick next process:** If no process is currently running:
 - Get all processes that have arrived (arrival time $\leq$ current time) and are not completed.
 - From these, find the process with the **lowest priority number** (highest priority).
 - If two processes have the same priority, the first one found is selected.
 - Assign it to the CPU.
3. **No process available?** If:
 - No process has arrived yet,
 - or all arrived processes are completed,the CPU remains idle for this tick

Code

```
class PriorityNonPreemptive(Scheduler):

    def __init__(self):
        super().__init__("Priority Non-Preemptive")

    def _get_highest_priority(self, current_time):
        arrived = self.get_arrived_processes(current_time)

        if not arrived:
            return None

        highest_prior = arrived[0]
        for p in arrived:
            if p.get_priority() < highest_prior.get_priority():
                highest_prior = p
        return highest_prior

    def get_next_process(self, current_time) -> Optional[Process]:

        if self.current_process is not None:
            if self.current_process.is_completed():
                self.current_process = None

            else:
                return self.current_process

        next_proc = self._get_highest_priority(current_time)
        return next_proc
```

Shortest-Job First (Preemptive)

Abstract

Also known as Shortest Remaining Time First, it is a preemptive version of the standard SJF algorithm, which can switch mid-execution to a more attractive process. A more attractive process is the one with a shorter remaining time than the current one.

This algorithm is based on the *greedy technique*, which focuses on taking the best local decision, which is the process with the shortest remaining time. It is better than the non-preemptive version but has the inherent starvation problem and introduces process swapping overhead.

Algorithm

1. Gather all arrived and not completed jobs (ready jobs).
2. Order these jobs by remaining job time.
 - a. If two jobs have the same remaining processing time, prioritize arrival time (first-come first-served).
 - b. If two jobs have the same remaining processing and arrival times, prioritize process id (*pid*).
3. Execute the job with the shortest remaining time (the front of the queue).
4. Repeat on process arrival.

Code

```
Algorithms > SRTF.py > ...
1  from Logic.scheduler import Scheduler
2  from Logic.process import Process
3  from typing import Optional
4
5  class SRTFScheduler(Scheduler):
6      def __init__(self):
7          super().__init__("Shortest Job First (Preemptive)")
8
9      def get_next_process(self, current_time: int) -> Optional[Process]:
10         ready_processes = self.get_arrived_processes(current_time)
11
12         if not ready_processes:
13             return self.current_process if not self.current_process.is_completed() else None
14
15         ready_processes.sort(key=lambda p: (
16             p.get_remaining_time(),
17             p.get_arrival_time(),
18             p.get_pid()
19         ))
20
21         shortest_ready = ready_processes[0]
22
23         if self.current_process and not self.current_process.is_completed():
24             current_is_shorter_or_equal = (
25                 self.current_process.get_remaining_time() <= shortest_ready.get_remaining_time()
26             )
27             return self.current_process if current_is_shorter_or_equal else shortest_ready
28
29         return shortest_ready
```

Shortest-Job First (Non-Preemptive)

Abstract

This scheduling algorithm prioritizes the shortest job time first to reduce and optimize waiting time. It is based on the Greedy technique that focuses on taking the best local decision, which is the job (process) with the shortest burst time. It is non-preemptive; when a process starts, it runs to completion.

In this implementation, the processes are assumed to have a known processing time. But in practice, the OS does not know the execution time of processes, it estimates them using exponential averaging:

$$\tau_{n+1} = \alpha t_n + (1 - \alpha)\tau_n$$

Its main drawback is **starvation**: a continuous stream of short processes can indefinitely delay longer ones.

Algorithm

1. Gather all arrived and not completed processes (ready jobs).
2. Order these processes by burst time.
 - a. If two jobs have the same burst time, prioritize arrival time (first-come first-served).
 - b. If two jobs have the same burst and arrival times, prioritize process id (*pid*).
3. Execute the shortest job (the front of the queue).
4. Repeat after completing any process.

Code

```
Algorithms > SJF.py > ...
1  from typing import List, Optional
2  from Logic.scheduler import Scheduler
3  from Logic.process import Process
4
5  class SJFScheduler(Scheduler):
6      def __init__(self):
7          super().__init__("Shortest Job First (Non-Preemptive)")
8
9      def get_next_process(self, current_time) -> Optional[Process]:
10         if self.current_process and not self.current_process.is_completed():
11             return self.current_process
12
13         ready_processes = self.get_arrived_processes(current_time)
14         if not ready_processes:
15             return None
16
17         # Sort list by burst time
18         # If two processes have same SJT, process the first
19         # If two processes have same arrival_time, process the lower pid
20         ready_processes.sort(key=lambda process: (process.get_burst_time(), process.get_arrival_time(), process.get_pid()))
21         return ready_processes[0]
```

Priority Preemptive Scheduling Algorithm

Abstract

Priority Preemptive (PP) is a CPU scheduling algorithm that selects the process with the highest priority (lowest priority number) from all arrived processes **every tick**. Unlike the non-preemptive version, if a new process arrives with a higher priority than the currently running process, the current process is **immediately interrupted** and the higher priority process takes over the CPU. This ensures that the most important process always runs first but may cause lower-priority processes to wait longer.

Our implementation extends an abstract Scheduler class and overrides a single method **get_next_process()**, which is called every tick to decide which process runs next. It uses a helper method **_get_highest_priority()** to find the best candidate from all arrived processes. Preemption happens automatically because **run_tick()** simply switches to whatever process is returned. It also supports dynamic process addition during runtime.

Algorithm

On **every tick**, the algorithm follows these steps:

1. Pick highest priority process:

- Get all processes that have arrived (arrival time $\leq$ current time) and are not completed.
- From these, find the process with the **lowest priority number** (highest priority).
 - If two processes have the same priority, the first one found is selected.
- Return it to the CPU.

2. Preemption:

- If the returned process is **different** from the currently running process, the current process is automatically preempted.
- No extra code is needed for this — **run_tick()** in the parent class simply switches to whatever process **get_next_process()** returns.

3. No process available? If:

- No process has arrived yet,
- or all arrived processes are completed,

the CPU remains idle for this tick.

Code

```
class PriorityPreemptive(Scheduler):

    def __init__(self):
        super().__init__("Priority Preemptive")

    def _get_highest_priority(self, current_time):
        arrived = self.get_arrived_processes(current_time)

        if not arrived:
            return None

        best = arrived[0]
        for p in arrived:
            if p.get_priority() < best.get_priority():
                best = p
        return best

    def get_next_process(self, current_time) -> Optional[Process]:
        next_proc = self._get_highest_priority(current_time)
        return next_proc
```

Priority Non-Preemptive Scheduling Algorithm

Abstract

Priority Non-Preemptive (PNP) is a CPU scheduling algorithm that selects the process with the highest priority (lowest priority number) from all arrived processes. Once a process starts running on the CPU, it is **never interrupted**. It runs to completion before the next process is selected. This ensures simplicity but may cause lower-priority processes to wait longer.

Our implementation extends an abstract Scheduler class and overrides a single method `get_next_process()`, which is called every tick to decide which process runs next. It uses a helper method `_get_highest_priority()` to find the best candidate from all arrived processes and supports dynamic process addition during runtime.

Algorithm

On **every tick**, the algorithm follows these steps:

1. **Check current process:** If there is a process currently running on the CPU, check two cases:
 - **Case A — Completed:** If the process has finished (remaining time = 0) clear it from the CPU.
 - **Case B — Still Running:** If the process has not finished, keep running it. **Do not interrupt it**, even if a higher priority process has arrived.
2. **Pick next process:** If no process is currently running:
 - Get all processes that have arrived (arrival time $\leq$ current time) and are not completed.
 - From these, find the process with the **lowest priority number** (highest priority).
 - If two processes have the same priority, the first one found is selected.
 - Assign it to the CPU.
3. **No process available?** If:
 - No process has arrived yet,
 - or all arrived processes are completed,the CPU remains idle for this tick.

Code

```
class PriorityNonPreemptive(Scheduler):

    def __init__(self):
        super().__init__("Priority Non-Preemptive")

    def _get_highest_priority(self, current_time):
        arrived = self.get_arrived_processes(current_time)

        if not arrived:
            return None

        highest_prior = arrived[0]
        for p in arrived:
            if p.get_priority() < highest_prior.get_priority():
                highest_prior = p
        return highest_prior

    def get_next_process(self, current_time) -> Optional[Process]:

        if self.current_process is not None:
            if self.current_process.is_completed():
                self.current_process = None

            else:
                return self.current_process

        next_proc = self._get_highest_priority(current_time)
        return next_proc
```


Round Robin (configurable time quantum)

Abstract

Round Robin (RR) is a CPU scheduling algorithm that gives each process a fixed time slice called a time quantum. If a process doesn't finish within its quantum, it goes to the back of the ready queue and the next process runs. This ensures fair CPU distribution and prevents starvation.

Our implementation extends an abstract Scheduler class and overrides a single method `get_next_process()`, which is called every tick to decide which process runs next. It uses a FIFO queue (deque) to manage the ready queue and supports dynamic process addition during runtime

Algorithm

On every tick, the algorithm follows these steps:

1. **Check for new arrivals:** Scan all processes in the system. If a process has arrived (its arrival time $\leq$ current time),

- has not completed,
- is not already in the ready queue,
- is not the currently running process,

add it to the back of the ready queue.

2. **Handle current process:** If there is a process currently running on the CPU, check three cases:

- **Case A — Completed:** If the process has finished (remaining time = 0), clear it from the CPU and reset the quantum counter.
- **Case B — Quantum Expired:** If the process has used up its full quantum but is not yet finished, move it to the back of the ready queue and reset the quantum counter.
- **Case C — Continue Running:** If the process still has quantum time left, increment the quantum counter and let it keep running.

3. **Pick next process:**

If no process is currently running, take the process at the front of the ready queue and assign it to the CPU, starting its quantum counter at 1.

4. **Queue empty? -> If:**

- The ready queue is empty,
 - no process is running,
- the CPU remains idle for this tick.

Code

```
class RoundRobin(Scheduler):

    def __init__(self, quantum: int):
        super().__init__("Round Robin")
        self.quantum = quantum
        self.ready_queue = deque()
        self.current_quantum_used = 0

    def _add_new_arrivals(self, current_time):
        for p in self.processes:
            if (
                p.get_arrival_time() <= current_time
                and not p.is_completed()
                and p not in self.ready_queue
                and p is not self.current_process
            ):
                self.ready_queue.append(p)

    def get_next_process(self, current_time) -> Optional[Process]:

        self._add_new_arrivals(current_time)
        if self.current_process is not None:

            if self.current_process.is_completed():
                self.current_process = None
                self.current_quantum_used = 0

            elif self.current_quantum_used >= self.quantum:
                self.ready_queue.append(self.current_process)
                self.current_process = None
                self.current_quantum_used = 0

            else:
                self.current_quantum_used += 1
                return self.current_process

        if self.ready_queue:
            next_proc = self.ready_queue.popleft()
            self.current_quantum_used = 1
            return next_proc

        return None

    def reset(self):
        super().reset()
        self.ready_queue = deque()
        self.current_quantum_used = 0
```

Chapter 4 : Core

Overview

The Core Logic layer serves as the brain of the entire CPU Scheduler application. It is responsible for managing the simulation clock, tracking every process throughout its lifecycle, and acting as the bridge between the user interface and the scheduling algorithms. This layer is composed of four files — Simulation, Scheduler, Process, and Execution — each handling a distinct responsibility that together drive the simulation from start to finish. While the GUI handles what the user sees and the Algorithms handle scheduling decisions, the Core Logic is what keeps everything running, coordinated, and in sync.

Scheduler.py

1.Import block

```
from abc import ABC, abstractmethod
from typing import List, Optional
from Core import Process #src.models.process
```

- ABC stands for **Abstract Base Class**. It's a built-in Python feature that lets you create a class that **cannot be instantiated directly** — it must be inherited by another class. This is exactly what you want here because Scheduler on its own is just a blueprint, not a real algorithm.
- abstractmethod is a decorator that marks specific methods as **mandatory to implement** in every subclass. If a subclass forgets to implement it, Python throws an error the moment you try to create an object from that subclass.
- List and Optional are typing helpers. List[Process] means "a list that contains Process objects." Optional[Process] means "either a Process object or None." These don't affect how the code runs — they just make the code easier to read and help your IDE catch mistakes early.
- Process is imported from your Core module because the scheduler manages a list of Process objects and needs to know what they are.

Class definition

```
class Scheduler(ABC):

    def __init__(self, name):
        self.name = name
        self.time_slice = 1 # How many time units a process runs per tick. Round Robin will override this.
        self.processes: List[Process] = list()
        self.current_time = 0
        self.current_process: Optional[Process] = None # Whatever is currently on the CPU (None if idle)
        self.completed_processes: List[Process] = list()
```

By inheriting from ABC, this class becomes abstract. This one line enforces the rule that **nobody can do Scheduler()** directly. You must always do FCFS(), SJF(), etc. It's a design decision that forces every algorithm to follow the same structure.

This runs every time a subclass like FCFS() is created. It sets up everything the scheduler needs to operate:

- **self.name** — a label for the algorithm, useful for displaying in the GUI or debugging.
- **self.time_slice** = 1 — how many time units a process runs per tick. This defaults to 1 for every algorithm. Round Robin is the only one that will override this with the user's chosen quantum value.
- **self.processes** — the master list of every process added to this scheduler, whether waiting, running, or finished.
- **self.current_time** = 0 — the simulation clock. Starts at zero and advances every tick. Every decision in the simulation is made relative to this value.
- **self.current_process** = None — whatever process is currently occupying the CPU. Starts as None because nothing is running yet. Goes back to None whenever the CPU is idle.
- **self.completed_processes** — a separate list that collects processes once they finish. Used for metrics calculation and for checking if everything is done.

2.Process adder & getter

```
def add_process(self, process: Process) -> None:      # Add a single process to the scheduler's list.
    self.processes.append(process)

def add_processes(self, processes: list[Process]) -> None:
    self.processes.extend(processes)

def get_processes(self) -> list[Process]:
    # Returns the full list of processes, including ones still running and ones already completed.
    return self.processes
```

- Two simple list operations. `add_process` adds one process at a time — this is what gets called when the user adds a process mid-simulation.
- `add_processes` adds a whole list at once using **extend()** this is used at the start when the user enters all their processes before hitting Start. Having both avoids writing a loop every time you want to add many processes.
- A straightforward getter. Returns the full process list including processes that are still waiting, currently running, and already completed. The GUI might call this to populate the table at startup.

3.Reset

```
def reset(self):
    self.current_time = 0
    self.current_process = None
    self.completed_processes = list()

def hard_reset(self):
    self.reset()
    for process in self.processes:
        process.reset()
```

- A **soft reset** — only the scheduler's own runtime state is wiped. The clock goes back to zero, the current process is cleared, and the completed list is emptied. The processes themselves are completely untouched, so their data (waiting time, turnaround time, execution history) is still readable after this call. This is useful when you want to re-examine results without re-running.
- A **full reset** — calls `reset()` first to handle the scheduler's side, then loops through every single process and calls `process.reset()` on each one, restoring them all to their original untouched state. This is what `Simulation.reset()` calls when the user wants to completely restart from scratch.

4.Clock

```
def get_current_time(self) -> int:
    # Returns where the simulation clock currently stands.
    return self.current_time

def all_processes_completed(self) -> bool:
    # Returns True only when every single process has finished running.
    return all(process.is_completed() for process in self.processes)
```

- A simple getter for the simulation clock. This is called by **`Simulation.add_live_process()`** to know what time it currently is, so it can set the arrival time of a dynamically added process to right now.
- Uses Python's built-in **`all()`** function with a generator expression. It checks **`is_completed()`** on every process and **only returns True** if every single one returns True. If even one process is still running or waiting, this returns False. The simulation's while loop checks every tick as its stop condition

5.Ready Queue

```
def get_arrived_processes(self, current_time) -> List[Process]: # Ready queue
    # Every algorithm calls this to get the list of candidates for the next tick.
    return [
        p
        for p in self.processes
        if (p.get_arrival_time() <= current_time) and (not p.is_completed())
    ]
```

This is the **ready queue** — **the most important helper method for all algorithms**. It uses a list comprehension to filter the master process list down to only the processes that are both eligible and still needed:

- **`p.get_arrival_time() <= current_time`** — the process has already arrived now
- **`not p.is_completed()`** — the process hasn't finished yet

Every single algorithm calls this method inside their **`get_next_process()`** to get the candidates for the next tick. Without this, algorithms would have to write their own filtering logic every time.

6.Average Timing Metrics

```
def get_average_waiting_time(self):
    # Adds up the waiting time of every process and divides by the total count.
    # Returns 0 if there are no processes, to avoid a division-by-zero crash.
    if not self.processes:
        return 0.0
    sum_waiting_time = 0
    for process in self.processes:
        sum_waiting_time += process.get_waiting_time()
    no_of_processes = len(self.processes)
    return sum_waiting_time / no_of_processes

def get_average_turnaround_time(self):
    # Same idea as waiting time above – sum all turnaround times and average them.
    if not self.processes:
        return 0.0
    sum_turnaround_time = 0
    for process in self.processes:
        sum_turnaround_time += process.get_turnaround_time()
    no_of_processes = len(self.processes)
    return sum_turnaround_time / no_of_processes

def get_average_response_time(self):
    # Response time is only meaningful for processes that have already started,
    # so we filter to those first before calculating the average.
    responded = [p for p in self.processes if p.get_response_time() is not None]
    if not responded:
        return 0.0
    return sum(responded) / len(responded)

def calculate_metrics(self):
    # One convenient call that returns both averages as a tuple.
    # The GUI calls this at the end of the simulation to display the final stats.
    return (self.get_average_waiting_time(), self.get_average_turnaround_time())
```

- First checks if there are any processes at all — if the list is empty, it **returns 0.0 immediately to avoid a division-by-zero crash**. Otherwise, it loops through every process, accumulates their waiting times, then divides by the total count. The waiting time of each process was already calculated inside **process.execute()** when the process finished, so this is just reading and averaging those stored values.
- **get_average_turnaround_time()** Exact same structure as **get_average_waiting_time()** but reads **get_turnaround_time()** instead. Turnaround time is completion time minus arrival time — the total time a process spent in the system from arrival to finish.
- Response time is how long a process waited before getting the CPU for the very first time. Not every process may have started yet mid-simulation, so it first filters to only processes that have a response time recorded (is not None). If none have started yet it returns 0.0 safely. Note: there is a minor bug here — **sum(responded)** should be **sum(p.get_response_time() for p in responded)** since **responded** is a list of Process objects, not numbers.
- **Calculate_matrices()** :A convenience wrapper that calls both metric methods and returns them together as a tuple. The GUI calls this single method at the end of the simulation to get everything it needs for the stats display in one shot, instead of making two separate calls.

7.Process Lookup & Removal

```
def find_process_by_pid(self, pid: int) -> Optional[Process]:
    # Walks through the process list looking for a matching PID.
    # Returns the process if found, or None if nothing matches.
    for process in self.processes:
        if process.get_pid() == pid:
            return process
    return None

def remove_process(self, pid: int) -> None:
    # Finds the process by PID first, then removes it from both lists it might
    # appear in — the main process list and the completed list.
    process = self.find_process_by_pid(pid)
    if process in self.processes:
        self.processes.remove(process)
    if process in self.completed_processes:
        self.completed_processes.remove(process)
```

- A simple linear search through the process list. Loops one by one and returns the first process whose PID matches. If nothing is found after checking everyone, it returns None. This is used internally by `remove_process()` and can also be used by the GUI to look up a specific process by ID.
- First calls `find_process_by_pid()` to locate the process. Then checks if it actually exists in the process list before trying to remove it — this prevents a crash if an invalid PID is passed. It also checks if the process appears in the `completed_processes` list and removes it from there too, keeping both lists consistent. Safe to call even if the PID doesn't exist.

8.Core Abstract Method

```
@abstractmethod
def get_next_process(self, current_time) -> Optional[Process]:
    # This is the only method each algorithm MUST implement on its own.
    pass
```

- This is the most important method in the entire file. It's decorated with `@abstractmethod` which means it has no real implementation here — it's just a declaration of a contract.
- Every subclass (FCFS, SJF, Priority, Round Robin) **must** override this with their own logic, or Python will refuse to let you create an object from that class.
- This single method is where all the differences between algorithms live. Everything else in this file is shared infrastructure. Only this method changes from algorithm to algorithm:
 - FCFS returns the process with the earliest arrival time
 - SJF returns the process with the shortest burst time
 - Priority returns the process with the smallest priority number
 - Round Robin tracks whose turn it is in the queue

9.Heartbeat

```
def run_tick(self) -> Process:
    # This is the heartbeat of the scheduler — it runs once every second.
    # It asks the algorithm who goes next, runs that process for one time unit,
    # checks if it finished, then advances the clock.

    print("getting next process")
    next_process = self.get_next_process(self.current_time)

    time_used = self.time_slice # Default time used — may be less if process finishes early
    print("inside tick")

    if next_process:
        self.current_process = next_process
        # execute() returns how much time was actually used,
        time_used = self.current_process.execute(self.current_time, self.time_slice)

        if self.current_process.is_completed():
            # Move it to the completed list so metrics can be read later
            self.completed_processes.append(self.current_process)
    else:
        # No process is ready yet — CPU is idle this tick
        self.current_process = None

    # Always advance the clock, whether the CPU was busy or idle
    self.current_time += time_used

    return self.current_process
```

- **run_tick()** is the **heartbeat** of the entire scheduler — it gets called once every second by the simulation loop and represents everything that happens in one unit of simulation time.
- The moment it's called, it immediately asks the algorithm who should run right now by calling **get_next_process()** with the current clock value. This is the only moment where the algorithms layer gets involved — whatever FCFS, SJF, or Priority decides gets handed back here as **next_process**.
- Before doing anything with that result, it sets **time_used** to the default **time_slice** (which is 1 for most algorithms) as a safe starting value, because the actual time used might turn out to be less than a full slice if the process finishes early.
- Then it checks whether an algorithm returned to a process or not. If it did, it sets it as the **current_process** and calls **execute()** on it, passing the current time and the time slice. The **execute()** call does the real work inside the Process — it decrements the remaining time, records an Execution entry for the Gantt chart, and returns how much time was consumed. That returned value overwrites **time_used**, so if a process only had half a unit left, the clock will only move by half. After executing, it immediately checks if the process just crossed the finish line — if **is_completed()** returns True, the process gets moved into **completed_processes** so it can be read for metrics later. If on the other hand no process was returned by the algorithm — meaning nothing has arrived at the CPU yet — **current_process** is simply set to None, representing an idle CPU for this tick.
- Regardless of whether the CPU was busy or idle, the clock always moves forward at the end by exactly **time_used**. Finally, **current_process** is returned to whoever called **run_tick()** — which is the simulation loop — so it can be yielded straight to the GUI to update the Gantt chart and the remaining burst timetable for that tick.

Process.py

1. Introduction

The Process class represents a single process in a CPU scheduling simulation. It encapsulates all attributes and behaviors required to simulate process execution, including timing metrics, execution tracking, and scheduling-related properties.

2. Imports and Dependencies

The implementation depends on two external components:

- Execution: A custom class used to record execution intervals (start and end times).
- Optional (from the typing module): Used to indicate that a variable may hold a value or None.

```
from Core import Execution
from typing import Optional
```

3. Class Definition

The constructor initializes all process attributes, including basic inputs (PID, name, arrival time, burst time, and optional priority), which are stored as private variables. It also performs input validation to ensure correctness, raising an error if the burst time is not positive or if the arrival time is negative. Runtime values such as remaining time (initialized to burst time) and execution-related fields are set to default states. Additionally, performance metrics are initialized, and an empty execution history list is created to track execution intervals.

```
class Process:
    """
    Represents a process in the CPU scheduler simulation.
    """

    def __init__(self, pid: int, name: str, arrival_time: int, burst_time: int, priority=None):
        if burst_time <= 0:
            raise ValueError(f"burst_time must be > 0, got {burst_time}")
        if arrival_time < 0:
            raise ValueError(f"arrival_time must be >= 0, got {arrival_time}")

        self.__pid: int = pid
        self.__name: str = name
        self.__arrival_time: int = arrival_time
        self.__burst_time: int = burst_time
        self.__priority: Optional[int] = priority
        self.__remaining_time: int = burst_time
        self.__start_time: Optional[int] = None
        self.__completion_time: Optional[int] = None
        self.__waiting_time: int = 0
        self.__turnaround_time: int = 0
        self.__response_time: Optional[int] = None
        self.__execution_history: list[Execution] = list()
```

4. Reset Function

The **reset()** method restores the process to its initial state. This is particularly useful when running multiple scheduling algorithms on the same set of processes. It clears all runtime-related values while preserving the original process parameters.

```
def reset(self):
    """Reset the process state for a new simulation."""
    self.__remaining_time = self.__burst_time
    self.__start_time = None
    self.__completion_time = None
    self.__waiting_time = 0
    self.__turnaround_time = 0
    self.__response_time = None
    self.__execution_history = list()
```

5. Process State Management

The method **is_completed()** checks whether the process has finished execution by verifying if the remaining time is zero or less. This method is essential for controlling scheduling loops.

```
def is_completed(self):
    """Check if the process has completed execution."""
    return self.__remaining_time <= 0
```

6. Time Calculation Methods

Turnaround Time = Completion Time – Arrival Time

This is the total time from when the process arrives until it fully finished. It only calculates if completion_time is set (meaning the process finished).

Waiting Time = Turnaround Time – Burst Time

This means: out of all the time it spent in the system, how much of it was just sitting and waiting (not actually using the CPU), It only runs after turnaround_time is calculated.

```
def calculate_turnaround_time(self):
    """Calculate and set the turnaround time."""
    if self.__completion_time is not None:
        self.__turnaround_time = self.__completion_time - self.__arrival_time

def calculate_waiting_time(self):
    if self.__turnaround_time is not None:
        self.__waiting_time = self.__turnaround_time - self.__burst_time
```

7. Execution Mechanism

The **execute()** method simulates the execution of the process on the CPU. It is the core function of the class and performs the following steps:

- Checks if the process is already completed.
- Records the start time if this is the first execution.
- Calculates the response time as the difference between the first execution time and arrival time.
- Executes the process for a specified time quantum.
- Updates the remaining time.
- Records the execution interval using the Execution class.
- If execution completes, it sets the completion time and calculates turnaround and waiting times.

The method returns the actual execution time used, which allows the scheduler to update the global clock.

```
def execute(self, current_time: int, time_quantum: int = 1) -> int:
    if self.is_completed():
        return 0

    if self.__start_time is None:
        self.__start_time = current_time

    if self.__response_time is None:
        self.__response_time = current_time - self.__arrival_time

    execution_time = min(self.__remaining_time, time_quantum)
    self.__remaining_time -= execution_time

    self.__execution_history.append(
        Execution(start_time=current_time, end_time=current_time + execution_time)
    )

    if self.is_completed():
        self.__completion_time = current_time + execution_time
        self.calculate_turnaround_time()
        self.calculate_waiting_time()

    return execution_time
```

8. Cloning Functionality

Creates a fresh copy of the process with the same inputs but zero runtime state. Useful if you want to test the same set of processes under different algorithms without affecting the originals

```
def clone(self):
    """Create a fresh clone of this process with no runtime state."""
    return Process(
        pid=self.__pid,
        name=self.__name,
        arrival_time=self.__arrival_time,
        burst_time=self.__burst_time,
        priority=self.__priority,
    )
```

9. Accessor Methods (Getters)

The class provides multiple getter methods to access private attributes such as:

- Process ID
- Arrival time
- Burst time
- Remaining time
- Waiting, turnaround, and response times

This ensures encapsulation and controlled access to internal data.

```
def get_pid(self) -> int:
    return self.__pid

def get_name(self) -> str:
    return self.__name

def get_start_time(self) -> Optional[int]:
    return self.__start_time

def get_arrival_time(self) -> int:
    return self.__arrival_time

def get_waiting_time(self) -> int:
    return self.__waiting_time

def get_turnaround_time(self) -> int:
    return self.__turnaround_time

def get_completion_time(self) -> Optional[int]:
    return self.__completion_time

def get_response_time(self) -> Optional[int]:
    return self.__response_time

def get_execution_history(self) -> list[Execution]:
    return self.__execution_history

def get_priority(self) -> Optional[int]:
    return self.__priority

def get_burst_time(self) -> int:
    return self.__burst_time

def get_remaining_time(self) -> int:
    return self.__remaining_time

def get_progress(self) -> float:
```

10. Helper Methods

Additional utility methods include:

- **get_progress():** Returns the percentage of execution completed.
- **has_started():** Indicates whether the process has begun execution.

```
def get_progress(self) -> float:
    return (self.__burst_time - self.__remaining_time) / self.__burst_time

def has_started(self) -> bool:
    """Returns True if the process has received CPU time at least once."""
    return self.__start_time is not None
```

11. String Representations

Two methods are implemented for displaying process information:

- **__str__():** Provides a detailed, human-readable summary of the process state.
- **__repr__():** Provides a concise representation useful for debugging and displaying lists of processes.

```
def __str__(self):
    """Human-readable string representation for debugging."""
    status = (
        "COMPLETED"
        if self.is_completed()
        else f"RUNNING ({self.__remaining_time}/{self.__burst_time})"
    )
    output = [
        f"Process {self.__pid} ({self.__name}):",
        f"  Status:      {status}",
        f"  Arrival time:  {self.__arrival_time}",
        f"  Burst time:    {self.__burst_time}",
        f"  Priority:      {self.__priority}",
        f"  Waiting time:  {self.__waiting_time}",
        f"  Turnaround:    {self.__turnaround_time}",
        f"  Response time: {self.__response_time}",
    ]
    return "\n".join(output)

def __repr__(self) -> str:
    """Short representation useful when printing lists of processes."""
    return (
        f"Process(pid={self.__pid}, name='{self.__name}', "
        f"burst={self.__burst_time}, remaining={self.__remaining_time}, "
        f"arrival={self.__arrival_time})"
    )
```


Execution.py

1. Constructor & Validation

```
class Execution:
    def __init__(self, start_time: int, end_time: int):
        # Raises: ValueError: If start_time is negative or end_time <= start_time
        if start_time < 0:
            raise ValueError(f"start_time must be >= 0, got {start_time}")
        if end_time <= start_time:
            raise ValueError(
                f"end_time must be > start_time, got end={end_time}, start={start_time}"
            )

        self.__start_time: int = start_time
        self.__end_time: int = end_time
```

Every time a process runs for a time slice, one **Execution** object is created and stored in the process's history. This is the data that draws each bar on the Gantt Chart.

- **start_time** and **end_time** are the only two values this class ever holds — everything else is derived from them
- Both are stored as **private** (__) meaning nothing outside this class can modify them directly, only through the controlled setters
- The two **ValueError** guards run before anything is stored, catching impossible records before they corrupt the Gantt Chart — a negative start or an end before the start should never exist

2. Duration, Getters & Setters

```
def duration(self) -> int:
    #Returns how long this execution slice lasted.
    return self.__end_time - self.__start_time

def get_start_time(self) -> int:
    return self.__start_time

def set_start_time(self, start_time: int) -> None:
    if start_time < 0:
        raise ValueError(f"start_time must be >= 0, got {start_time}")
    if start_time >= self.__end_time:
        raise ValueError(
            f"start_time must be < end_time ({self.__end_time}), got {start_time}"
        )
    self.__start_time = start_time

def get_end_time(self) -> int:
    return self.__end_time

def set_end_time(self, end_time: int) -> None:
    if end_time <= self.__start_time:
        raise ValueError(
            f"end_time must be > start_time ({self.__start_time}), got {end_time}"
        )
    self.__end_time = end_time
```

- **duration()** is the most used method — it returns how wide each Gantt bar should be by simply subtracting start from end
- The two getters are the only safe doors to read the private values — used by the GUI, the Process class, and any algorithm that inspects execution history
- The setters carry the same validation logic as the constructor — they refuse any value that would create an impossible or zero-length time slice, keeping the data consistent even after creation

3. Time Slice Query Methods

```
def overlaps(self, other: "Execution") -> bool:
    # Checks if this execution slice overlaps in time with another one.
    return self.__start_time < other.get_end_time() and self.__end_time > other.get_start_time()
def contains(self, time_point: int) -> bool:
    #Returns True if the given time point falls within this execution slice.
    return self.__start_time <= time_point < self.__end_time
```

- overlaps() checks whether two execution slices share any time — useful for **validating** that the scheduler never accidentally ran two processes at the same moment, which would be a scheduling bug
- contains() checks if a specific point in time falls within this slice — useful for the GUI to **highlight** the currently active bar on the Gantt Chart as the simulation clock ticks forward
- Both are pure read-only checks — they never modify anything

4. Execution Class Dunder Methods

```
def __eq__(self, other: object) -> bool:
    # Two executions are equal if they cover the exact same time range.
    if not isinstance(other, Execution):
        return False
    return self.__start_time == other.get_start_time() and self.__end_time == other.get_end_time()

def __str__(self) -> str:
    # Human-readable format for debugging.
    return f"Execution[{self.__start_time} → {self.__end_time}] (duration={self.duration()})"

def __repr__(self) -> str:
    return f"Execution(start={self.__start_time}, end={self.__end_time})"
```

- __eq__ defines what "equal" means for two Execution objects — two slices are equal only if they cover the **exact same time range**. Without this, Python would compare by memory address, and two identical slices would never match
- __str__ is what gets printed in a human-readable way — useful when debugging the simulation output in the terminal
- __repr__ gives a shorter technical format like Execution(start=2, end=5) — this is what appears when you print a **list** of executions, making the process history readable immediately

Simulation.py

1. Class & Constructor

```
class Simulation:
    def __init__(self, scheduler: Scheduler, delay: float = 1.0):
        self.scheduler = scheduler
        self.delay = delay          # Seconds between ticks. Default is 1.0 (real-time, 1 unit = 1 second)
        self.running = False       # Flipped to True when the user hits Start
        self.paused = False        # Toggled by the pause/resume button
        self.processes_timeline = []
```

- **Simulation** is the top-level controller that the GUI talks to directly. It doesn't make any scheduling decisions itself — instead, it owns the scheduler and tells it when to tick, when to stop, and how fast to go.
- At creation it receives whichever algorithm the user selects and stores it as **self.scheduler**.
- The **delay** defaults to 1.0 second, meaning one unit of simulation time equals one real second.
- **running** and **paused** are the two state flags the GUI reads and flips to control the simulation's lifecycle.
- **processes_timeline** is declared here for future use but is not yet active.

2. Process Management

```
def add_process(self, process: Process):
    # Just passes the process down to the scheduler.
    self.scheduler.add_process(process)

def add_live_process(
    self, name: str, burst_time: int, priority: int, pid: int
) -> Process:
    # This is the "add while running" feature.
    # Instead of asking the user for an arrival time, we use the current simulation time automatically — so
    current_time = self.scheduler.get_current_time()

    process = Process(
        pid=pid,
        name=name,
        arrival_time=current_time, # Arrives right now, at this moment in the simulation
        burst_time=burst_time,
        priority=priority,
    )
    self.add_process(process)

    return process

def remove_process(self, pid: int):
    # Delegates removal down to the scheduler, which handles cleaning up the process from all internal lists
    self.scheduler.remove_process(pid)
```

These three methods are how processes enter and leave the simulation.

1. **add_process()** is the standard path used at startup — it simply passes the process down to the scheduler.
2. **add_live_process()** is the dynamic injection feature — instead of asking the user for an arrival time, it reads the current simulation clock and uses that as the arrival time automatically, so the new process joins the ready queue on the very next tick.
3. **remove_process()** delegates straight down to the scheduler which handles cleaning up all internal lists safely.

3.Simulation Control

```
def reset(self):
    # Wipes everything – resets the scheduler clock and restores
    self.scheduler.hard_reset()

def is_running(self) -> bool:
    # The GUI checks this to know whether the simulation is currently active.
    return self.running

def start(self):
    # Called when the user clicks "Start". Marks the simulation as running and makes sure it's not
    self.running = True
    self.paused = False

def is_paused(self) -> bool:
    # The GUI checks this to decide whether to show "Resume" or "Pause".
    return self.paused

def set_paused(self, paused: bool):
    # Toggles the pause state. The simulation loop checks this flag each tick to decide whether
    self.paused = paused
```

These are the control methods the GUI buttons map to directly.

- **start()** arms the simulation by flipping **running** to True and ensuring it is not stuck in a paused state from a previous run.
- **set_paused()** toggles the pause flag which the simulation loop checks on every tick.
- **reset()** delegates to **hard_reset()** on the scheduler, which wipes the clock and restores every process back to its original state.
- **is_running()** and **is_paused()** are the read-only checks the GUI polls to decide what to display at any given moment.

3.Speed Control

```
def set_speed(self, speed_factor: int):
    # Adjusts how fast the simulation runs by changing the delay between ticks.
    # Speed 1x → 1.0s delay (real time)
    # Speed 2x → 0.5s delay
    # Speed 4x → 0.25s delay
    # Speed 0 or below → near-instant (0.001s), used as a safe lower bound
    if speed_factor <= 0:
        self.delay = 0.001
    else:
        self.delay = 1.0 / speed_factor
```

Controls how fast the simulation runs by adjusting the delay between ticks. The math is simple — speed 1x gives a 1.0s delay, speed 2x gives 0.5s, speed 4x gives 0.25s. If the factor is zero or negative, the delay is set to 0.001 seconds as a safe lower bound to prevent a division-by-zero crash. Setting this to effectively zero is also how the At-Once mode achieves its instant results.

4. The Simulation Loop

```
def _run_simulation(self, useDelay: bool = True):
    # Instead of running the whole simulation at once and blocking everything
    # Setting useDelay=False skips the sleep entirely, giving instant results
    # That's the "At-Once" mode – same logic, just no waiting between ticks.

    print("Starting tick loop")
    while (self.running) and (not self.scheduler.all_processes_completed()):
        print("Running Tick...")
        current_process = self.scheduler.run_tick()

        if useDelay:
            # Wait one real second (or however long the delay is set to) before
            time.sleep(self.delay)

        print("Yielding current process")
        # Hand the current state back to the GUI so it can update the visuals
        # Execution will resume here on the next iteration.
        yield current_process
        print("After yield")

    # Loop ended – either the user stopped it or all processes finished.
    self.running = False
    return self.running
```

- This is **the most critical method** in the entire file — it is a **Python Generator**, meaning it uses `yield` instead of `return`.
- Rather than running the whole simulation at once and freezing the interface, it runs one tick at a time and hands the result back to the GUI after every tick so it can update the Gantt chart and the table before the next tick begins.
- The while loop keeps going as long as the simulation is marked as running and there are still unfinished processes.
- Each iteration calls `run_tick()` on the scheduler — which advances the clock by one unit — then optionally sleeps for **`self.delay`** seconds to create the real-time feel.
- The `yield` is what makes this a generator — it pauses the function and delivers `current_process` to the GUI, then resumes exactly where it left off on the next call. When **`useDelay=False`**, the sleep is skipped entirely and the loop runs at maximum speed, which is the **At-Once mode** that produces instant results.

Chapter 5 : GUI

1. main_window.py

Abstract:

The Graphical User Interface (GUI) serves as the interactive front-end for the CPU scheduling simulation, engineered using the PyQt5 framework. It provides a robust platform for managing processes, configuring algorithms, and visually tracking execution via an animated Gantt chart. The architecture relies on seamless integration with a backend RunController to decouple simulation logic from the visual layer, ensuring real-time UI updates and data integrity. The system supports multiple scheduling algorithms (FCFS, SJF, SRTF, Priority, Round Robin) and dynamically reconfigures its input fields—such as conditionally exposing time quantum or priority parameters—based on the active algorithm selection.

System Architecture & Initialization:

- **Window & Layouts:** Instantiates the main application window using responsive layout managers to organize the dashboard, control panels, and visualization areas.
- **Component Construction:** Initializes all core PyQt5 widgets, including input fields (line edits), control buttons, the Process Table, and the Gantt chart drawing canvas.
- **Event Binding:** Establishes Signals and Slots to connect UI interactions (like button clicks or dropdown changes) to their corresponding backend controller methods.

Dynamic Algorithm Selection

- **UI Adaptation:** Detects selection changes and dynamically reconfigures input fields to prevent invalid data entry.
- **Parameter Configuration:** Automatically enables the 'Time Quantum' input exclusively for Round Robin and activates the 'Priority' field only for priority-based algorithms.

```
def _on_algo_changed(self, text: str):
    is_round_robin = "Round Robin" in text
    is_priority_algo = "Priority" in text

    self.lblQuantum.setEnabled(is_round_robin)
    self.spinQuantum.setEnabled(is_round_robin)
    self.lblPriority.setEnabled(is_priority_algo)
    self.spinPriority.setEnabled(is_priority_algo)
```

Process Management & Table Integration

The Process Table (built via QTableWidgetItem) acts as the central data hub, strictly enforcing data types to ensure the RunController receives valid parameters.

- **Add/Remove Process:** Captures and validates user inputs (Arrival, Burst, Color). It stores the process in the backend memory while simultaneously updating the visual table.

```
def _add_row_to_proc_table(self, data: dict, note: str = ""):
    row = self.procTable.rowCount()
    self.procTable.insertRow(row)
    pri = str(data["priority"]) if data["priority"] is not None else "-"
    arrival = str(data["arrival"]) if data["arrival"] >= 0 else "live"
    for col, val in enumerate([str(data["pid"]), data["name"] + note, arrival, str(data["burst"]), pri]):
        self.procTable.setItem(row, col, QTableWidgetItem(val))
```

- **Inject Live:** Supports dynamic scheduling by allowing users to add processes mid-simulation, immediately syncing new data with the active RunController.
- **Pre-Simulation View:** Displays initial user-defined parameters such as Arrival Time, Burst Time, and Priority.

Simulation Control & Execution

Execution Modes

- **Run Live:** Triggers a time-driven, step-by-step execution that visualizes context switching tick-by-tick.

```
def _on_inject_live(self):
    current_time = self._controller.get_scheduler().get_current_time()

    name = self.edtName.text().strip() or f"P{self._next_pid}"
    priority = self.spinPriority.value() if "Priority" in self.comboAlgo.currentText() else 0
    data = {
        "pid": self._next_pid,
        "name": name,
        "arrival": current_time,
        "burst": self.spinBurst.value(),
        "priority": priority,
    }
    self._process_store.append(data)
    self._add_row_to_proc_table(data, note=" (live)")
    self.edtName.clear()
    self._next_pid += 1
    self._controller.inject_process(
        name=data["name"], burst=data["burst"],
        priority=priority, pid=data["pid"],
    )
```

- **Run At Once:** Bypasses the timer to instantly compute the entire schedule and render the final results.

Operational States

- Includes Pause/Resume functionality to control the simulation clock and a Reset feature to flush all backend structures and clear the UI components.

Real-Time Monitoring & Metrics

- **Visual Tracking:** Dynamically appends color-coded blocks to the Gantt Chart at each CPU tick.

- **Dynamic Synchronization:** Continuously refreshes the 'Remaining Time' column in the table, allowing users to track preemption and progress in real time.
- **Statistical Output:** Upon process completion, the system populates the table with calculated performance metrics: Finish Time, Turnaround Time (TAT), Waiting Time (WT), and Response Time (RT).

2. run_controller.py

Defines two classes — **Worker** and **RunController** — that together form the **concurrency and simulation control layer** of the CPU Scheduling Simulator, acting as the bridge between the PyQt5 GUI and the underlying scheduling logic.

Worker Class.

Worker is a lightweight QObject designed to execute the simulation loop off the main GUI thread. It holds a reference to a Simulation instance and a use_delay flag that determines whether the simulation runs with real-time pacing (live mode) or as fast as possible (instant mode). Its run() method iterates over the generator produced by Simulation._run_simulation(), and on each yielded process emits the sig_tick signal carrying the process object and its (start, end) time window. Once the generator is exhausted, it emits sig_done to signal completion. The separation of Worker from RunController is a deliberate Qt threading pattern — objects must be moved to a thread rather than subclassing QThread directly, keeping the design clean and safe.

RunController Class

RunController is the orchestration hub that the MainWindow interacts with directly. It exposes three outward-facing PyQt5 signals: sig_tick (relayed from the worker for each scheduling step), sig_done (simulation completion), and sig_status (human-readable status strings like "running live", "paused", or "finished") used to update the GUI's status label. The static factory method make_scheduler() acts as a **strategy selector**, mapping algorithm name strings to concrete scheduler instances (FCFS, SJF, SRTF, Priority Non-Preemptive, Priority Preemptive, Round Robin), decoupling algorithm instantiation from the rest of the system.

```

def make_scheduler(algo: str, quantum: int = 2):
    if algo == "FCFS":
        return FCFSScheduler()
    elif algo == "SJF (Non-Preemptive)":
        return SJFScheduler()
    elif algo == "SRTF (Preemptive SJF)":
        return SRTFScheduler()
    elif algo == "Priority (Non-Preemptive)":
        return PriorityNonPreemptive()
    elif algo == "Priority (Preemptive)":
        return PriorityPreemptive()
    elif algo == "Round Robin":
        return RoundRobin(quantum=quantum)

    raise ValueError("Unknown algorithm")

```

Simulation Lifecycle

The `_setup()` method converts raw process dictionaries from the GUI into Process domain objects, registers them with the chosen scheduler, wraps everything in a Simulation instance, and calls `sim.start()`. The `_launch()` method then creates a QThread, moves the Worker onto it, wires the thread's started signal to `worker.run()`, and connects the worker's signals upward to the controller — establishing a clean, non-blocking execution pipeline. The `run_live()` and `run_at_once()` public methods are the only two entry points for triggering a simulation, differing solely in the `use_delay` flag passed to the worker.

```

def _launch(self, use_delay: bool):

    self.thread = QThread()
    self.worker = Worker(self.sim, use_delay)

    self.worker.moveToThread(self.thread)

    self.thread.started.connect(self.worker.run)

    self.worker.sig_tick.connect(self.sig_tick)
    self.worker.sig_done.connect(self._on_done)
    self.worker.sig_done.connect(self.thread.quit)

    self.thread.start()

```

Runtime Control

`pause()` and `resume()` delegate directly to `Simulation.set_paused()`, allowing the GUI to freeze and unfreeze the scheduling loop mid-execution without terminating the thread. `stop()` forcefully halts the simulation by setting `sim.running = False` and calls `thread.quit()` with a 2-second wait timeout to ensure graceful thread teardown. The `inject_process()` method enables the live injection feature: it validates that the simulation is still active via `is_running()`, then calls `sim.add_live_process()` to dynamically insert a new process into the running scheduler, immediately emitting a status update with the current simulation time — enabling real-time, interactive scheduling experiments without restarting the simulation.

3. gantt_widget.py

This file implements the **real-time Gantt Chart visualization component** of the CPU Scheduling Simulator using PyQt5's custom painting system. It is split into two classes following a classic **canvas/viewport pattern**: GanttCanvas handles all rendering logic, while GanttChart wraps it inside a scrollable container.

Layout Constants:

Five module-level constants define the chart's spatial grammar: CELL_W = 36 (pixel width per time unit), ROW_H = 34 (height of each process row), HEADER_H = 24 (height of the time-axis header), and LABEL_W = 70 (width of the process name label column on the left). A palette of eight distinct hex color strings in COLORS is used to visually differentiate processes from one another in a round-robin assignment pattern.

GanttCanvas — State & Data Model

GanttCanvas extends QWidget and maintains five internal state variables: `_slots` (a list of (name, start, end) tuples representing each CPU execution window), `_row_order` (a list of process names in first-appearance order, determining vertical row positions), `_color_map` (a dictionary mapping process names to their assigned color), `_color_idx` (the rolling index for color assignment), and `_total_time` (the rightmost time boundary, driving canvas width). The `add_slot()` method is the sole data ingestion point — it registers new processes into the color map and row order on first encounter, appends the slot, updates `_total_time`, and calls both `_resize()` and `update()` to trigger an immediate repaint. The `clear()` method resets all five fields back to their initial state.

```
def add_slot(self, process, start: int, end: int):
    name = process.get_name() if process else None
    if name and name not in self._color_map:
        self._color_map[name] = COLORS[self._color_idx % len(COLORS)] # assign color to the process in order of appearance
        self._color_idx += 1
    if name and name not in self._row_order:
        self._row_order.append(name) # add to row order if not already present
    self._slots.append((name, start, end))
    self._total_time = end
    self._resize()
    self.update()
```


GanttCanvas — Rendering Pipeline

All drawing is handled inside `paintEvent()`, which orchestrates three sequential passes through a `QPainter`. First, `_draw_header()` renders the time-axis ruler: it draws tick marks at every time unit and numeric labels at every fifth unit (plus zero), giving the chart a clean, readable time scale. Second, `_draw_rows()` fills alternating row backgrounds, renders the left-side process label for each entry in `_row_order`, draws a subtle vertical separator between the label column and the chart area, and overlays a vertical grid line for every time unit across the full row height. Third, `_draw_blocks()` iterates over `_slots` and paints each execution block as a `QRect` with a semi-transparent fill (alpha 80) derived from the process's assigned color, a solid 1px border in the same color, and a centered name label — but only if the block is wide enough (more than 20 pixels). Idle CPU slots (where name is `None`) are rendered distinctly with a light grey fill and the text "idle", always placed on the first row.

GanttChart — Scrollable Wrapper

`GanttChart` extends `QScrollArea` and serves as the public-facing component used by `MainWindow` and `RunController`. It wraps `GanttCanvas` with `setWidgetResizable(False)` so the canvas controls its own dimensions, enforces a horizontal scrollbar that is always visible, and sets a minimum height of 140px. Its `add_slot()` method delegates to the canvas and then immediately scrolls the horizontal scrollbar to its maximum value — ensuring the chart always **auto-scrolls to show the latest scheduling activity** as the simulation progresses in real time.

```
class GanttChart(QScrollArea):
    """The class the controller and main window interact with."""

    def __init__(self, parent=None):
        super().__init__(parent)
        self._canvas = GanttCanvas()
        self.setWidget(self._canvas)
        self.setWidgetResizable(False)
        self.setMinimumHeight(140)
        self.setHorizontalScrollBarPolicy(Qt.ScrollBarAlwaysOn)
        self.setVerticalScrollBarPolicy(Qt.ScrollBarAsNeeded)

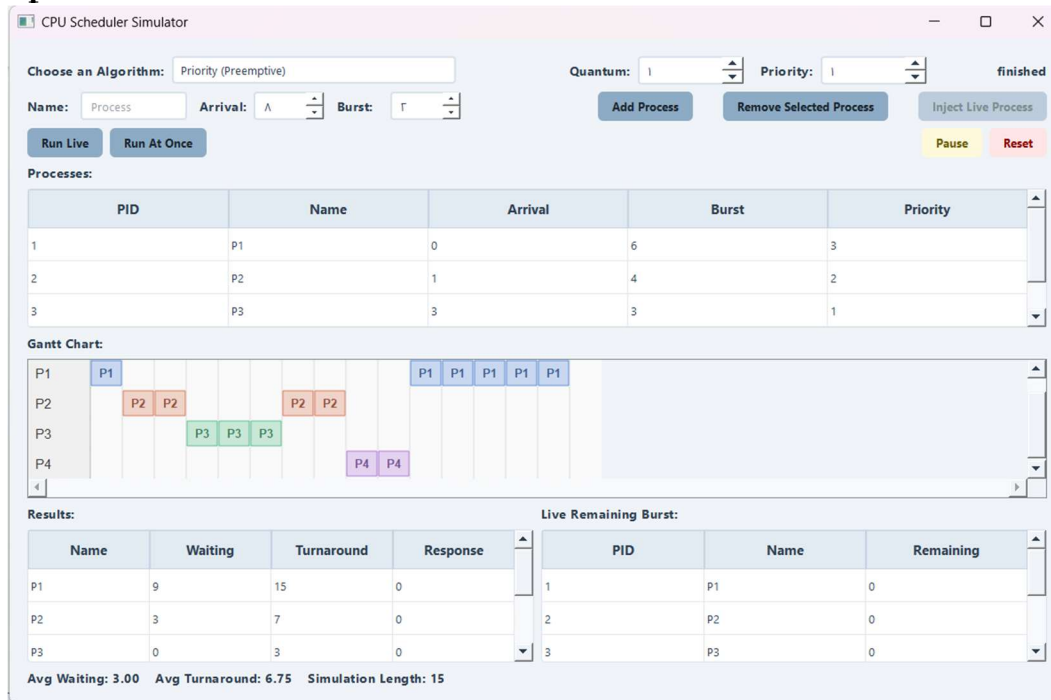
    def clear(self):
        self._canvas.clear()

    def add_slot(self, process, start: int, end: int):
        self._canvas.add_slot(process, start, end)
        self.horizontalScrollBar().setValue(self.horizontalScrollBar().maximum())
```

Chapter 6 : Final Output

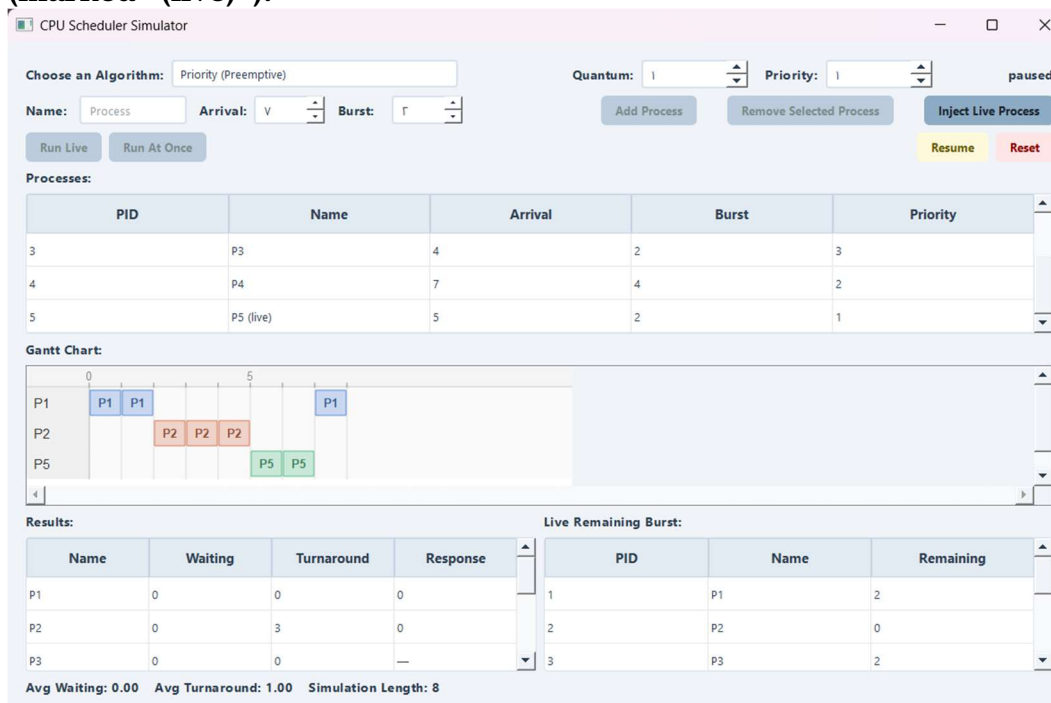
Priority Scheduling (regular run)

Priority (Preemptive) execution – Gantt chart, metrics, and remaining burst table update live.



Dynamic Addition (inject while paused)

Paused simulation – priority spinbox re-enabled; injecting a new live process (marked “(live)”).



Idle Time Case

FCFS with idle gap (P1 finishes at 3, next arrives at 5) – grey “idle” block appears in Gantt chart.

